

SHUTTLE BOT

Platform Layanan Mahasiswa via Telegram

Telegram Bot

Next.js Dashboard

PostgreSQL

TypeScript

Dokumentasi resmi mencakup arsitektur sistem, cara kerja, panduan pengguna, prosedur instalasi, konfigurasi lingkungan, dan referensi API.

Versi Dokumen	1.0.0
Tanggal	Mei 2026
Stack Utama	GrammY · Express · Next.js · Prisma · PostgreSQL
Lisensi	Internal / Campus Use

DAFTAR ISI

1. Gambaran Umum Aplikasi

- Apa itu Shuttle Bot?
- Fitur Utama
- Komponen Sistem

2. Arsitektur & Cara Kerja

- Arsitektur Sistem
- Alur Kerja Pengguna (User Flow)
- Alur Kerja Freelancer
- Sistem Matching & Kontrak
- Risk Score & Sanksi Otomatis

3. Panduan Pengguna (User Guide)

- Pengguna / Customer
- Freelancer
- Admin Dashboard

4. Instalasi & Setup

- Prasyarat
- Clone & Install Dependencies
- Konfigurasi Environment (.env)
- Setup Database (Prisma)
- Menjalankan Aplikasi

5. Referensi API

- Authentication
- Freelancer Endpoints
- Order Endpoints
- Report & Stats Endpoints
- WebSocket

6. Testing

- E2E Tests (Playwright)
- Stress Tests (Artillery)

7. Deployment

- Mode Produksi
- Variabel Lingkungan Produksi

1. GAMBARAN UMUM APLIKASI

1.1 Apa itu Shuttle Bot?

Shuttle Bot adalah platform layanan berbasis Telegram yang dirancang khusus untuk ekosistem mahasiswa kampus. Platform ini menghubungkan pengguna (mahasiswa/user) dengan penyedia jasa (freelancer) untuk tiga jenis layanan utama: antar-jemput, jastip (jasa titip belanja), dan jasa lainnya. Sistem ini terdiri dari dua komponen utama: sebuah Telegram Bot yang menjadi antarmuka utama pengguna dan freelancer, serta sebuah Admin Dashboard berbasis web untuk pengelolaan operasional oleh tim manajemen kampus.

Seluruh transaksi dijamin keamanannya melalui sistem verifikasi identitas (KTM/KTP), kontrak digital otomatis, rating & ulasan, serta mekanisme laporan dan sanksi yang beroperasi secara otomatis.

1.2 Fitur Utama

Fitur	Keterangan
Layanan Antar Jemput (Anjem)	Pemesanan ojek/taksi khusus kampus dengan estimasi harga oleh user
Layanan Jastip	Titip belanja di toko dalam/sekitar kampus dengan multi-lokasi
Layanan Jasa Lainnya	Berbagai jenis jasa custom yang bisa dipesan melalui bot
Sistem Matching Otomatis	Bot mencocokkan pesanan ke freelancer aktif secara real-time
Kontrak Digital Otomatis	Setiap pesanan yang diterima menghasilkan kontrak hash unik
Verifikasi KTM/Selfie	Upload foto KTM dan selfie saat pendaftaran freelancer
Rating & Ulasan	User dapat memberi rating 1-5 dan komentar setelah pesanan selesai
Sistem Laporan (Report)	User bisa melaporkan freelancer dengan bukti foto
Risk Score Otomatis	Skor risiko freelancer dihitung berdasarkan laporan valid & rating
Sanksi Otomatis	Sistem memberikan peringatan/suspend otomatis berdasarkan laporan
Admin Dashboard Real-time	Panel web dengan statistik live via WebSocket
Broadcast Pesan	Admin dapat broadcast pesan ke semua freelancer aktif
Multi-kampus Ready	Dapat dikonfigurasi untuk berbagai institusi via environment variables

1.3 Komponen Sistem

Aplikasi terdiri dari tiga bagian utama yang bekerja bersama:

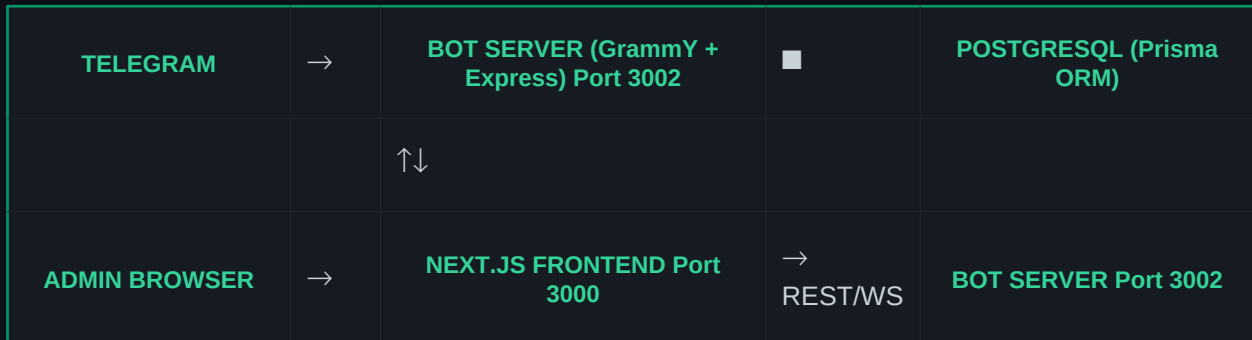
Komponen	Teknologi	Fungsi
----------	-----------	--------

Backend + Telegram Bot	Node.js · TypeScript · Grammy · Express	API server, bot logic, matching, notifikasi
Frontend Admin	Next.js 14 · TypeScript · Tailwind CSS · Recharts	Dashboard web untuk monitoring & manajemen
Database	PostgreSQL · Prisma ORM	Penyimpanan data users, orders, reports, sessions
Penyimpanan Berkas	Cloudinary	Upload & hosting foto KTM, selfie, bukti laporan
Real-time Channel	WebSocket (ws)	Push update statistik live ke dashboard admin

2. ARSITEKTUR & CARA KERJA

2.1 Arsitektur Sistem

Shuttle Bot menggunakan arsitektur monorepo di mana backend (bot + REST API) dan frontend (admin dashboard) dijalankan secara terpisah namun terhubung melalui REST API dan WebSocket.



2.2 Alur Kerja Pengguna (User Flow)

Langkah	Aksi Pengguna	Yang Terjadi di Sistem
1	/start di Telegram	Bot auto-register user ke database (upsert berdasarkan Telegram ID)
2	Pilih layanan (Anjem/Jastip/Jasa)	Bot menampilkan menu inline keyboard
3	Isi form pesanan (lokasi, detail, harga)	Data disimpan ke session Prisma, lalu order dibuat di DB
4	Order dibuat	Sistem langsung mencoba matching ke freelancer tersedia
5	Freelancer ditemukan	Bot notifikasi ke freelancer, user diberi info freelancer (60 detik)
6	Freelancer terima / tolak	Jika terima → order RUNNING; jika tolak/timeout → re-match
7	Pesanan selesai (DONE)	User diminta memberi rating 1-5 dan komentar
8	Rating dikirim	Sistem update avgRating freelancer & recalculate risk score

2.3 Alur Kerja Freelancer

Langkah	Aksi	Keterangan
---------	------	------------

1	Daftar Freelancer	Upload KTM + selfie, isi kontak darurat → status PENDING
2	Verifikasi Admin	Admin approve via dashboard → status APPROVED, bot kirim notifikasi
3	Terima Notifikasi Order	Bot kirim detail pesanan + tombol Terima/Tolak
4	Terima dalam 60 detik	Status order berubah jadi MATCHED, kontrak digital dibuat
5	Jalankan pesanan	Tekan tombol Mulai → status RUNNING
6	Selesaikan pesanan	Tekan tombol Selesai → status DONE, user diminta rating
7	Lihat profil	Freelancer bisa cek statistik, rating, dan riwayat order

2.4 Sistem Matching & Kontrak Digital

Ketika sebuah pesanan masuk, sistem secara otomatis menjalankan proses matching:

- Sistem mencari freelancer dengan status APPROVED yang tidak sedang menangani order aktif.
- Freelancer yang ditemukan mendapat notifikasi Telegram dengan tombol Terima/Tolak.
- Jika freelancer tidak merespons dalam 60 detik, sistem secara otomatis reset order ke WAITING dan mencoba matching ulang ke freelancer lain.
- Setelah freelancer menerima, sistem membuat Kontrak Digital berisi hash unik (berdasarkan order ID + freelancer ID + timestamp) dan detail lengkap pesanan dalam format JSON.
- Kontrak ini berfungsi sebagai bukti kesepakatan yang tidak bisa diubah (immutable).

2.5 Risk Score & Sanksi Otomatis

Sistem secara otomatis menghitung dan memperbarui Risk Score freelancer berdasarkan formula:

$$\text{Risk Score} = \min(100, \text{laporan_valid} \times 25 + (5 - \text{avg_rating}) \times 5)$$

Kondisi	Sanksi Otomatis	Durasi
1-2 laporan valid	Peringatan (WARNING)	Permanen (catatan)
3+ laporan valid	Suspend (SUSPEND_7D)	7 hari (auto-lift setelah habis)
Keputusan admin	Ban Permanen (BANNED)	Permanen

3. PANDUAN PENGGUNA (USER GUIDE)

3.1 Panduan untuk Pengguna / Customer

Membuat Pesanan:

- 1. Mulai Bot:** Buka Telegram, cari bot sesuai nama kampus, kirim perintah /start.
- 2. Pilih Layanan:** Tap tombol layanan yang diinginkan: Antar Jemput, Jastip, atau Jasa Lainnya.
- 3. Isi Form:** Ikuti panduan bot, isi lokasi jemput/tujuan, detail, dan estimasi harga yang kamu tawarkan.
- 4. Konfirmasi:** Bot akan menampilkan ringkasan pesanan. Tap Konfirmasi untuk mengirim pesanan.
- 5. Tunggu Matching:** Bot akan mencari freelancer. Kamu akan dinotifikasi saat freelancer ditemukan.
- 6. Pantau Status:** Status pesanan dapat dipantau dari menu utama bot.
- 7. Beri Rating:** Setelah pesanan selesai, bot akan meminta rating 1-5 dan komentar opsional.

Membatalkan Pesanan:

- Pesanan hanya bisa dibatalkan selama masih berstatus WAITING (belum ada freelancer).
- Tap tombol "Batalkan Pesanan Aktif" di menu utama, lalu konfirmasi pembatalan.

Melaporkan Freelancer:

- Setelah pesanan selesai, pilih "Laporkan Freelancer" dari riwayat pesanan.
- Isi deskripsi masalah dan upload foto bukti (opsional).
- Laporan akan diproses oleh admin dan dapat mengakibatkan sanksi pada freelancer.

3.2 Panduan untuk Freelancer

Pendaftaran Freelancer (5 Langkah):

- Langkah 1:** Tap "Daftar sebagai Freelancer" di menu utama bot.
- Langkah 2:** Kirim foto KTM atau KTP yang jelas (tidak buram/gelap).
- Langkah 3:** Kirim foto selfie sambil memegang KTM/KTP.
- Langkah 4:** Masukkan nama lengkap kontak darurat.
- Langkah 5:** Masukkan nomor HP kontak darurat. Pendaftaran terkirim, tunggu verifikasi admin (1-24 jam).

Menerima & Menyelesaikan Order:

- Bot mengirim notifikasi dengan detail pesanan dan tombol Terima/Tolak.
- Kamu punya 60 detik untuk merespons. Jika tidak ada respons, pesanan diteruskan ke freelancer lain.
- Setelah menerima, tap "Mulai" untuk mengubah status menjadi RUNNING.
- Setelah menyelesaikan layanan, tap "Selesai" untuk menutup pesanan.

- Rating dari user akan mempengaruhi Risk Score dan reputasimu.

3.3 Panduan Admin Dashboard

Dashboard admin dapat diakses melalui browser di URL yang dikonfigurasi (default: `http://localhost:3000`). Masuk menggunakan username dan password yang telah ditetapkan di konfigurasi server.

Menu	URL	Fungsi
Overview	<code>/dashboard</code>	Statistik real-time: total user, freelancer, order, grafik tren, status aktif
Freelancers	<code>/dashboard/freelancers</code>	Kelola freelancer: approve, suspend, ban, lihat profil detail & riwayat
Orders Log	<code>/dashboard/orders</code>	Monitor semua pesanan dengan filter status, tipe, tanggal, pencarian
Incidents	<code>/dashboard/reports</code>	Proses laporan user: validasi, invalidasi, tambah catatan admin
Broadcast	<code>/dashboard/broadcast</code>	Kirim pesan ke semua freelancer aktif (APPROVED) via Telegram

Catatan: Dashboard menggunakan WebSocket untuk memperbarui statistik secara real-time tanpa perlu refresh halaman. Indikator status server di sidebar menunjukkan apakah backend sedang online.

4. INSTALASI & SETUP

4.1 Prasyarat

Software	Versi Minimum	Keterangan
Node.js	≥ 18.x	Runtime JavaScript server-side
npm / pnpm	≥ 8.x	Package manager
PostgreSQL	≥ 14.x	Database relasional
Git	Terbaru	Version control
Akun Telegram	-	Untuk membuat bot via @BotFather
Akun Cloudinary	Free tier cukup	Penyimpanan foto KTM/selfie/bukti laporan

4.2 Clone & Install Dependencies

```
# 1. Clone repository
git clone shuttle-bot
cd shuttle-bot

# 2. Install dependencies Backend
npm install

# 3. Install dependencies Frontend (di folder terpisah)
cd frontend # atau nama folder Next.js-mu
npm install
cd ..
```

4.3 Konfigurasi Environment (.env)

Buat file .env di root folder backend berdasarkan template di bawah ini. Semua variabel bertanda * wajib diisi.

Variabel	Contoh / Default	Keterangan
BOT_TOKEN *	1234567890:ABCdef...	Token bot Telegram dari @BotFather
DATABASE_URL *	postgresql://user:pass@localhost:5432/shuttledb	Koneksi PostgreSQL (Prisma format)
JWT_SECRET *	minimal-32-char-random-string	Secret untuk signing JWT admin
ADMIN_USERNAME	admin	Username login dashboard (default: admin)

ADMIN_PASSWORD *	SuperSecret123!	Password admin, minimal 8 karakter
PORT	3002	Port HTTP server Express
NODE_ENV	development	development / production / test
WEBHOOK_URL	https://domain.com	URL publik untuk webhook Telegram (produksi)
BOT_NAME	Shuttle Bot	Nama platform yang tampil di bot
CAMPUS_NAME	UNNES	Singkatan nama kampus
CAMPUS_CITY	Semarang	Kota kampus
PICKUP_EXAMPLE	Gedung A, Sekaran	Contoh lokasi jemput untuk hint di form
STORAGE_PREFIX	unnes-shuttle	Prefix folder Cloudinary per kampus
DRIVER_ZONES	Sekaran,FT,FIS,FIK	Zona driver (pisah koma)

Untuk frontend Next.js, buat file .env.local dengan isi:

```
NEXT_PUBLIC_API_URL=http://localhost:3002/api
```

4.4 Setup Database (Prisma)

```
# Buat database di PostgreSQL
createdb shuttledb

# Jalankan migrasi Prisma (buat semua tabel)
npx prisma migrate dev --name init

# Atau untuk produksi (deploy tanpa create shadow db)
npx prisma migrate deploy

# Generate Prisma Client
npx prisma generate

# (Opsional) Buka Prisma Studio untuk inspeksi data
npx prisma studio
```

4.5 Menjalankan Aplikasi

Development Mode (Long Polling):

```
# Terminal 1 – Backend + Bot
npm run dev

# Output: [Bot] @namabot started (long polling)
# Output: [API] http://localhost:3002/api

# Terminal 2 – Frontend Dashboard
cd frontend
npm run dev
```

```
# Output: ready - started server on http://localhost:3000
```

Production Mode (Webhook):

```
# Set di .env:
```

```
NODE_ENV=production
```

```
WEBHOOK_URL=https://yourdomain.com
```

```
# Build dan jalankan
```

```
npm run build && npm start
```

5. REFERENSI API

Semua endpoint API berada di base URL: `http://localhost:3002/api` Kecuali endpoint login, semua endpoint memerlukan header Authorization: Bearer .

5.1 Authentication

Method	Endpoint	Body	Response
POST	/auth/login	{ username, password }	{ token, expiresIn: "7d" }

5.2 Freelancer Endpoints

Method	Endpoint	Keterangan
GET	/freelancers	Daftar freelancer (filter: status, minRisk, maxRisk, search, page, limit)
GET	/freelancers/:id	Detail freelancer + sanctions + reports + order history
PATCH	/freelancers/:id/approve	Approve freelancer (kirim notif Telegram)
PATCH	/freelancers/:id/suspend	Suspend freelancer (body: { reason, adminNote })
PATCH	/freelancers/:id/ban	Ban permanen freelancer (body: { reason, adminNote })
GET	/freelancers/:id/stats	Statistik detail freelancer (rating, orders, dsb.)

5.3 Order Endpoints

Method	Endpoint	Keterangan
GET	/orders	Daftar order (filter: status, type, date, search, page, limit)
GET	/orders/:id	Detail order + user + freelancer + contract + status logs + reports

5.4 Report & Stats Endpoints

Method	Endpoint	Keterangan
GET	/reports	Daftar laporan (filter: status, freelancerId, page, limit)
PATCH	/reports/:id/validate	Validasi laporan (body: { adminNote })
PATCH	/reports/:id/invalidate	Invalidasi laporan (body: { adminNote })
GET	/stats	Statistik overview: total users, freelancers, orders per tipe & status

POST	/broadcast	Broadcast pesan ke semua freelancer (body: { message })
------	------------	---

5.5 WebSocket

Dashboard terhubung ke WebSocket untuk update real-time. Koneksi dilakukan dengan:

```
ws://localhost:3002/api/ws?token=
```

Event (type)	Payload	Keterangan
STATS_UPDATE	{ type, data: StatsResponse }	Dikirim server setiap ada perubahan order/freelancer

6. TESTING

6.1 E2E Tests (Playwright)

Pengujian end-to-end dilakukan menggunakan Playwright untuk mensimulasikan interaksi browser nyata pada admin dashboard.

Instalasi & Menjalankan:

```
# Install Playwright
npx playwright install

# Jalankan semua test
npx playwright test

# Jalankan dengan UI mode
npx playwright test --ui
```

Test Case	Deskripsi
Security Redirect	User tidak ter-autentikasi diarahkan ke halaman login saat akses /dashboard
Auth & Login Flow	Login dengan kredensial valid, redirect ke dashboard
Navigasi Sidebar	Klik menu Freelancers, Orders, Incidents, Broadcast → halaman berubah sesuai
API Integration Tests	Uji semua endpoint REST: auth, freelancers, orders, reports, broadcast, health check

6.2 Stress Tests (Artillery)

Pengujian beban dilakukan menggunakan Artillery untuk memastikan API dapat menangani traffic tinggi di lingkungan kampus.

Instalasi & Menjalankan:

```
# Install Artillery secara global
npm install -g artillery

# Jalankan stress test
ADMIN_USERNAME=admin ADMIN_PASSWORD=pass artillery run stress/api.stress.yml --output report.json

# Generate HTML report
artillery report report.json
```

Phase	Durasi	Request/detik	Tujuan
-------	--------	---------------	--------

Warm-up	30 detik	10 req/s	Pemanasan server, identifikasi baseline
Ramp-up	60 detik	10 → 100 req/s	Simulasi traffic meningkat bertahap
Peak Load	60 detik	100 req/s	Simulasi traffic puncak kampus
Spike	15 detik	200 req/s	Simulasi lonjakan mendadak
Cool-down	30 detik	10 req/s	Pemulihan server

Kriteria lulus: p95 latency < 2000ms dan error rate < 1%. Jika tidak memenuhi kriteria, Artillery akan exit dengan kode error.

7. DEPLOYMENT

7.1 Mode Produksi

Pada mode produksi, bot menggunakan Webhook (bukan long polling) untuk menerima update dari Telegram. Ini lebih efisien dan disarankan untuk server publik.

- 1. Pastikan server memiliki domain publik dengan HTTPS (misalnya via Nginx + Certbot).
- 2. Set `NODE_ENV=production` dan `WEBHOOK_URL=https://yourdomain.com` di file `.env`.
- 3. Bot akan otomatis mendaftarkan webhook URL ke Telegram saat startup.
- 4. Build frontend: `npm run build`, lalu deploy ke server atau Vercel/Netlify.
- 5. Gunakan PM2 atau systemd untuk menjaga proses backend tetap berjalan.

Contoh setup dengan PM2:

```
# Install PM2
npm install -g pm2

# Build TypeScript
npm run build

# Jalankan dengan PM2
pm2 start dist/index.js --name shuttle-bot

# Simpan konfigurasi PM2 (auto-restart saat reboot)
pm2 save && pm2 startup
```

7.2 Variabel Lingkungan Produksi Penting

Variabel	Nilai Produksi	Alasan
NODE_ENV	production	Aktifkan webhook mode, disable debug logs
WEBHOOK_URL	https://yourdomain.com	URL publik HTTPS server
JWT_SECRET	Random 64+ karakter	Keamanan token admin
ADMIN_PASSWORD	Strong password	Cegah akses tidak sah ke dashboard
DATABASE_URL	Connection string prod DB	Pastikan gunakan database terpisah dari dev